

Implantación de aplicaciones web

Bloque 2:

Programación web: back-end

PHP

Profesor titular de Formación Profesional

Jorge Horcas Pulido

Posgraduado en Ing. informática

jorge.horcas@doc.medac.es

Índice

1. Definición
2. Sintaxis básica
3. Ejemplo
4. Variables
5. Salidas (output)
6. Tipos de datos
7. Cadenas de caracteres (string)
8. Números
9. Operadores
10. Arrays
11. Condicionales
12. Bucles
13. Funciones
14. Actividades finales

1. Definición

PHP es un lenguaje de programación interpretado del lado del servidor utilizado en el desarrollo web para realizar efectos dinámicos e interactivos en la web. Por ello:

- Para ejecutarlo es necesario que el archivo/s php se encuentren alojados en el servidor
- Los archivos php se escriben con extensión **.php**

2. Sintaxis básica

- Declaración: php empieza con la sintaxis **<?php** y termina con **?>**
- Cada línea de programación termina con **;**
- Los comentarios se escriben con **//** para una línea y **/* */** para varias líneas
- Se puede combinar con etiquetas **<html>**

3. Ejemplo

```
<html>
  <body>
    <h1>Pagina PHP</h1>

    <?php
      echo "Hola mundo";
    ?>
  </body>
</html>
```

```
// Ejemplo de PHP con salida de texto en HTML
<?php
  ECHO "<h1>Hola mundo</h1>";
?>
```

4. Variables

- Empiezan con el signo
- El primer carácter del nombre de la variable puede ser una letra o `_`
- Los nombres de las variables sólo pueden tomar (A-z, 0-9, and `_`)
- Son sensibles a mayúsculas y minúsculas (`$edad` y `$EDAD` are son variables diferentes)
- Las variables en php **no necesitan una declaración del tipo de dato para definir las**. Ya que php asocia automáticamente el tipo de datos a la variable

```
<?php
$cadena="Jorge"
ECHO "Mi nombre es $cadena";
?>
```

```
<?php
$num1=5;
$num1=4;
ECHO $num1 + $num2 ;
?>
```

- Para saber el tipo de datos asociado a la variable podemos usar la función `var_dump()`

```
// Visualización del tipo de dato de la variable
<?php
$num=5;
$cadena="Jorge";
var_dump($num);      // int(5)
var_dump($cadena);   // string(5) "Jorge"
?>
```

- Las variables declaradas pueden funcionar de forma: `global` o `local`

5. Salidas (output)

- Para visualizar por pantalla podemos usar 2 expresiones muy similares **echo** o **print**

```
// opciones de output
```

```
<?php
```

```
$texto1 = "PHP";
```

```
$texto2 = "Nebrija";
```

```
$x = 5;
```

```
$y = 4;
```

```
echo "<h2>" . $texto1 . "</h2>";
```

```
echo "Estudio en la" . $texto2 . "<br>";
```

```
echo $x + $y;
```

```
?>
```

```
// opciones de output
```

```
<?php
```

```
$texto1 = "PHP";
```

```
$texto2 = "Nebrija";
```

```
$x = 5;
```

```
$y = 4;
```

```
print "<h2>" . $texto1 . "</h2>";
```

```
print "Estudio en la" . $texto2 . "<br>";
```

```
print $x + $y;
```

```
?>
```

6. Tipos de datos

- String
- Integer
- Float
- Boolean
- Array
- Object
- NULL
- Resource

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<pre>
```

```
<?php
```

```
var_dump(5);
```

```
var_dump("John");
```

```
var_dump(3.14);
```

```
var_dump(true);
```

```
var_dump([2, 3, 56]);
```

```
var_dump(NULL);
```

```
?>
```

```
</pre>
```

```
</body>
```

```
</html>
```

```
int(5)
```

```
string(4) "John"
```

```
float(3.14)
```

```
bool(true)
```

```
array(3) {
```

```
    [0]=>
```

```
    int(2)
```

```
    [1]=>
```

```
    int(3)
```

```
    [2]=>
```

```
    int(56)
```

```
}
```

7. Cadenas de caracteres (string)

- Se pueden usar “ ” o ‘ ’

```
$cadena = "Jorge";  
$cadena = 'Jorge';
```

- Para realizar un salto de línea usamos dentro de la cadena `/n`

```
$cadena1 = "Instituto FP";  
$cadena2 = "\nNebrija";
```

- Para unir cadenas se usa el signo `.`

```
$nombre = "Jorge";  
$centro = "Nebrija";  
echo "Mi nombre es ".$nombre." y estudio en ".$centro ;
```

- También es posible incluir las variables dentro de una misma cadena usando “ ”

```
$nombre = "Jorge";  
$centro = "Nebrija";  
echo "Mi nombre es $nombre y estudio en $centro" ;
```

Nota: Si en vez de usar “ ” usamos ‘ ’ no se referencia la variable y considera \$nombre o \$centro como texto literal

- Podemos añadir más cadenas de caracteres en una variable declarada usando `.=`

```
$nombre = "Jorge";  
$nombre .= " Horcas";
```

- Principales **funciones con cadenas**:

- Tamaño de una cadena `strlen("cadena")`
- Número de palabras de una cadena `str_word_count("cadena")`
- Buscar un texto dentro de la cadena `strpos("cadena","palabra")`

- Principales funciones para **modificar cadenas**

- Convertir minúsculas en mayúsculas `strtoupper("cadena")`
- Convertir mayúsculas en minúsculas `strtolower("cadena")`
- Reemplazar caracteres dentro de la cadena `str_replace("palabra","palabrar","cadena")`

- Principal función para **subdividir cadenas**

- Reemplazar caracteres dentro de la cadena `substr("cadena",posición_inicial,num_posicones)`

Posición inicial: si es positiva empieza por el inicio de la cadena (la primera posición es 0). Si es negativa empieza por el final de la cadena (la primera posición es -1).

Actividades de control

1. Crea una variable llamada alumno que incluya tu nombre
2. Añade en la misma variable tu apellido
3. Averigua cuántos caracteres y cuántas palabras tiene tu nombre
4. Guarda en una variable distintas tu nombre desde la variable alumno
5. Ahora guarda en una variable distinta tu apellido, pero empezando por el final de la cadena
6. Declara una variable llamada archivo que incluya el valor 'foto.jpg'
Busca la extensión del archivo

8. Números

- Tipos de números **integer**, **float** o **number string**

```
$a = 5;  
$b = 5.34;  
$c = "25";
```

- Para verificar si es verdadero o falso el tipo de número se pueden usar las funciones:
 - `var_dump(is_int())`
 - `var_dump(is_float())`
 - `var_dump(is_numeric())`
- Para convertir de un tipo numérico a otro

```
// opciones de output  
<?php  
    // Convertir de decimal a entero  
    $x = 23465.768;  
    $entero = (int)$x;  
    echo $entero;  
  
    // Convertir de decimal a entero  
    $x = "23465.768";  
    $entero = (int)$x;  
    echo $entero;  
?>
```

9. Operaciones

- Definir constantes: **define**(*name, value, case-insensitive*)
 - name*: nombre de la constante
 - value*: valor de la constante
 - case-insensitive*: si no distingue entre mayúsculas o minúsculas. Por defecto toma el valor **false**
- Las operaciones aritméticas que se pueden realizar con números

Operation:		Example:
Addition	+	<code>echo 1 + 4.5; // Prints: 5.5</code>
Subtraction	-	<code>echo 9 - 1; // Prints: 8</code>
Multiplication	*	<code>echo -1.9 * 2.9; // Prints: -5.51</code>
Division	/	<code>echo 9 / 1; // Prints: 9</code>
Modulo	%	<code>echo 11 % 3; // Prints: 2</code>
Exponentiation	**	<code>echo 8**2; // Prints: 64</code>

- Si en una operación se incluye la misma variable `$sum = $sum+1` podemos simplificarlo:

Operation:	Long Syntax:	Short Syntax:
Add	<code>\$x = \$x + \$y</code>	<code>\$x += \$y</code>
Subtract	<code>\$x = \$x - \$y</code>	<code>\$x -= \$y</code>
Multiply	<code>\$x = \$x * \$y</code>	<code>\$x *= \$y</code>
Divide	<code>\$x = \$x / \$y</code>	<code>\$x /= \$y</code>
Mod	<code>\$x = \$x % \$y</code>	<code>\$x %= \$y</code>

Actividades de control

1. Programa el siguiente juego de magia con números:
 - a. Crea una variable llamada 'num' y asígnale un valor numérico. Para realizar el truco de magia debemos asegurarnos que el número siempre es un número entero
 - b. Crea una variable llamada 'respuesta' y asígnale la variable num
 - c. Suma al resultado 2
 - d. Multiplica el resultado por 2
 - e. Resta al resultado 2
 - f. Divide el resultado entre 2
 - g. Resta al resultado el número de inicio
 - h. ¡¡¡El resultado siempre tiene que ser 1!!!

10. Arrays

a. Arrays indexados

- Declaración de un array: `$nombre = array(valor1, valor2, valor3,...)`

```
$alumno = array("Juan", 18, "DAM");
```

También se puede declarar directamente entre `[]`

```
$coche = ["BMW", "Toyota", "Nisan"];
```

Nota: Se puede utilizar como elementos del arrays cualquier tipo de datos reconocidos en PHP

- Para visualizar un valor del arrays escribimos el nombre del array y entre `[]` la posición del valor que queremos recuperar. La primera posición empieza en 0

```
$coche = ["BMW", "Toyota", "Nisan"];  
echo $coche[0] // recupera el valor BMW  
echo $coche[1] // recupera el valor Toyota  
echo $coche[2] // recupera el valor Nisan
```

- Para imprimir todos los valores del array se puede usar la función `print_r($nombre_array)`

b. Arrays asociativos

- Un arrays asociativo es un arrays donde las posiciones del arrays toman nombres que nosotros asignamos

- Declaración de un array asociativo:

```
$nombre = array("indice1"=>valor1, "indice2"=>valor2, "indice3"=>valor3, ...)
```

```
$coche = array("marca"=>"Ford", "modelo"=>"Orion", "color"=>"rojo");
```

- Para visualizar un valor del arrays escribimos el nombre del array entre [] y el nombre que le hemos asociado a ese valor

```
echo $coche["marca"] // recupera el valor Ford
echo $coche["modelo"] // recupera el valor Orion
echo $coche["color"] // recupera el valor rojo
```

c. Arrays multidimensionales

- Un array multidimensional es un array que contiene uno a más arrays

- Ejemplo de array multidimensional en 2 dimensiones

```
$nombre = array(array(valor1, valor2, ...), array(valor1, valor2, ...), ...)  
  
$coche = array (array("Ford","rojo"), array("BMW","azul"), array("Land Rover","blanco"));
```

- Para visualizar un valor del array multidimensional debemos escribir la posición en 2 dimensiones, donde en el primer [] indicamos la posición de los elementos del array principal y en el segundo [] indicamos la posición de los elementos que se incluyen dentro de los arrays del array principal

```
echo $coche[0][0] // recupera el valor Ford  
echo $coche[0][1] // recupera el valor rojo  
echo $coche[1][0] // recupera el valor BMW  
echo $coche[1][1] // recupera el valor azul
```

Actividades de control

1. Crea un array formado por 3 elementos. Los 2 primeros tienen que ser números y el tercero una función que devuelva la suma de ellos.
2. Crea un array asociativo con los datos de un alumno que incluya nombre, apellido, edad, teléfono, correo, centro, ciclo, curso.
3. Crea un array multidimensional llamado alumno que codifique como primer elemento del array el nombre del alumno y el ciclo de informática que está matriculado y en los otros elementos el nombre de las asignaturas con su nota. Los arrays definidos dentro del array alumno son asociativos.

11. Condicionales

- Definir condicionales:
 - `if`: ejecuta un bloque de código si una condición es verdadera
 - `if...else`: ejecuta un bloque de código si una condición es verdadera y otro código si esa condición es falsa
 - `if...elseif...else`: ejecuta códigos diferentes para más de dos condiciones

Operadores comparativos

Operator	Name
<code>==</code>	Equal
<code>===</code>	Identical
<code>!=</code>	Not equal
<code><></code>	Not equal
<code>!==</code>	Not identical
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

Operadores lógicos

Operator	Name
<code>and</code>	And
<code>&&</code>	And
<code>or</code>	Or
<code> </code>	Or
<code>xor</code>	Xor
<code>!</code>	Not

- **Ejemplo 1**

Usando la función `date("H")` que nos indica la hora del sistema programar con condicionales que dependiendo de la hora el muestre el mensaje: “buenos días”, “buenas tardes”, “buenas noches”

```
$hora = date("H");

if ($hora > "0" && $hora <="12") {
    echo "Buenos días";
} elseif ($hora >= "13" && $hora <="20") {
    echo "Buenas tardes";
} else {
    echo "Buenas noches";
}
```

- **Ejemplo 2**

Comprobar que un número es un número primo comprendido entre el 1 y 10

```
$num=3;
if ($num==1 || $num==3 || $num==5 || $num==7){
    echo "$num es un número primo del 1 al 10";
}else{
    echo "$num no es un número primo del 1 al 10";
}
```

- Definir condicionales:

- `switch`: para cada valor de una variable se ejecuta un bloque de código

```
switch (variable) {  
  case valor1:  
    //código;  
    break;  
  case valor2:  
    //código;  
    break;  
  default:  
    //código;  
}
```

- **Ejemplo 3**

El usuario tiene que elegir entre 3 colores: rojo, azul o verde. De lo contrario tiene que mostrar un mensaje de que no ha elegido ninguno de los 3 colores

```
echo "Elige un color: rojo,azul,verde";
$color = "rojo";

switch ($color) {
case "rojo":
    echo "Ha elegido el color rojo";
break;
case "azul":
    echo "Ha elegido el color azul";
break;
case "verde":
    echo "Ha elegido el color verde";
break;
default:
    echo "Error: elija un color: rojo,azul,verde";
}
```

Actividades de control

1. Realizar con condicionales un programa que dado 3 números determine cuál es el mayor y cuál es el menor de los 3.
2. La función `date("w")` permite obtener los días de la semana. Programar con condicionales un programa que para cada valor me muestre por pantalla el nombre del día de la semana.

12. Condicionales

- **Definir condicionales:**

- **while:** repite en bucle el código incluido mientras la condición sea verdadera

```
while (condición) {  
    //código;  
}
```

- **Ejemplo 1**

Imprime por pantalla los 10 primeros números naturales

```
$num=1;  
while ($num <= 10){  
    echo $num."\n";  
    $num++;  
}
```

- **Ejemplo 2**

Calcula la suma de los 10 primeros números naturales

```
$num=1;  
$suma=0;  
while ($num <= 10){  
    $suma = $suma + $num;  
    $num ++;  
}  
echo $suma."\n";
```

- **Definir bucles:**

- **do...while**: se ejecuta el código al menos una vez y se repite en bucle mientras la condición sea verdadera

```
do {  
    //código;  
} while (condición);
```

- **Ejemplo 3**

Imprime los números múltiplos de 2 hasta un número máximo

```
$num=2;  
$max=10;  
do{  
    echo " $num \n";  
    $num+=2;  
}while($num<$max);
```

- **Definir bucles:**

- **for**: repite en bucle el código incluido el número de veces indicado

```
for (valor_inicial, valor_final, incremento) {  
    //código;  
}
```

- **Ejemplo 4**

Imprime por pantalla los 10 primeros números naturales

```
for($n=1; $n<=10; $n++){  
    echo $n."\n";  
}
```

- **Ejemplo 5**

Calcula la suma de los 10 primeros números naturales

```
$suma=0;  
for($n=1; $n<=10; $n++){  
    $suma+=$n;  
}  
echo $suma;
```


- **foreach**: repite en bucle los elementos de un array

```
$nombre_array = array(valor1, valor2, valor3, ...);  
foreach ($nombre_array as $x) {  
    //código;  
}
```

- **Ejemplo 6**

Imprime por pantalla los valores del array diccionario indicando el número de la palabra

```
$diccionario=array("abaco", "barco", "casa");  
$num_palabra=0;  
foreach($diccionario as $palabra){  
    $num_palabra++;  
    echo"palabra".$num_palabra." : ". $palabra." \n";  
}
```

- **foreach**: repite en bucle los elementos de un array asociativos

```
$nombre_array = array("indice1"=>valor1, "indice2"=>valor2, "indice3"=>valor3, ...);  
foreach ($nombre_array as $x => $y) {  
    //código;  
}
```

- **Ejemplo 7**

Imprime por pantalla los valores del array diccionario con su definición indicando el número de la palabra

```
$diccionario=array(  
    "abaco"=>"instrumento aritmetico",  
    "barco"=>"vehiculo maritimo",  
    "casa"=> "vivienda");  
$num_palabra=0;  
foreach($diccionario as $palabra => $definicion){  
    $num_palabra++;  
    echo $num_palabra.":".$palabra."> ".$definicion."\n";  
}
```

Actividades de control

1. Crea con `while` un programa que imprima todos los números impares del 1 al 50.
2. Crea con `do...while` un programa que controle el crecimiento del cultivo de hortalizas. Actualmente las hortalizas tienen una altura de 5cm. Cada semana crecen a un ritmo de 5cm. A partir de una altura de 20cm están listas para ser recolectadas.

Queremos que:

- a. Cada semana me notifique el crecimiento de las hortalizas
 - b. Que me notifique cuando están disponibles para ser recolectadas
3. Crea con `for` que muestre una cuenta atrás del 10 al 1 y en la que para los 3 últimos números en vez de mostrar el número muestre el mensaje “preparados”, “listos”, “ya”
 4. Usando `foreach` crea un array con los nombres y las notas finales de los alumnos y me lo muestre en forma de listado. Separa el nombre de la nota con “ : ”

13. Funciones

- **Sintaxis básica funciones sin parámetros**

- Para declarar la función se utiliza la declaración `function`

```
function nombreFunción() {  
    //código;  
}
```

- Para llamar a la función

```
nombreFunción();
```

- **Ejemplo 1**

Declara una función que muestre un mensaje por pantalla

```
function Mensaje() {  
    echo "Bienvenido";  
}  
Mensaje();
```

- **Sintaxis básica funciones con parámetros**

- Para declarar la función con parámetros, los valores se incluyen en los paréntesis

```
function nombreFunción($parametro1,$parametro2, ...) {  
    //código;  
}  
nombreFunción(valor1,valor2, ...);
```

- **Ejemplo 2**

Declara una función que muestra los nombres de los miembros de una familia

```
function familiaGarcia($nombre) {  
    echo "$nombre Garcia<br>";  
}  
familiaGarcia("Javier");  
familiaGarcia("Ana");  
familiaGarcia("Juan");  
familiaGarcia("Maria");
```

- **Ejemplo 3**

Declara una función que muestra los nombres de los miembros de una familia y su fecha de nacimiento

```
function familiaGarcia($nombre,$año) {  
    echo "$nombre Garcia. Nacido en $año<br>";  
}  
familiaGarcia("Javier",1980);  
familiaGarcia("Ana",1979);  
familiaGarcia("Juan",1983);  
familiaGarcia("Maria",1985);
```

- **Sintaxis básica funciones con un valor de parámetro por defecto**

- Para declarar un valor de parámetro por defectos se pone el valor entre signo =

```
function nombreFunción($parametro=valor) {  
    //código; }
```

- **Ejemplo 4**

Declara una función que muestra la altura. Si no se muestra un valor poner poder defecto 160

```
function medirAltura($altura = 160) {  
    echo "Su altura es : $altura <br>";  
}  
medirAltura(180);  
medirAltura(); // toma el valor 160
```

- **Sintaxis básica funciones con parámetros de número variable**

- Declarar el parámetro variable al final poniendo `...`

```
function nombreFunción(...$parametro) {  
    //código;  
}  
  
nombreFunción(valor1,valor2, ...);
```

- **Ejemplo 5**

Declara una función que sume n números

```
function sumarNumeros(...$n) {  
    $suma = 0;  
    $len = count($n);  
    for($i = 0; $i < $len; $i++) {  
        $suma += $n[$i];  
    }  
    echo $suma;  
}  
  
sumarNumeros(1,2,3,4,5,6,7,9);
```

- **Funciones return**

- Son funciones que retornan un valor declarando al final **return**

```
function nombreFunción($parametro) {  
    //código;  
    return $parametro }  
nombreFunción(valor1);
```

- **Ejemplo 6**

Declara una función que retorne la suma de 2 números

```
function sum($num1, $num2) {  
    $suma = $num1 + $num2;  
    return $suma;  
}  
echo "5 + 10 = " . sum(5, 10) . "<br>";
```


Actividades de control

1. Crea una función llamada `Familia()` que tome como valor de parámetros el apellido de la familia y el nombre de todos sus miembros. La función debe devolver con `return` un texto con el número total de miembros seguido de su nombre y apellido por línea.

2. Crea una función llamada `Aleatoria()` que:

a. Tome como valores un intervalo de números donde el primer parámetro es el número inferior y el segundo parámetro el número superior. La función debe devolver con `return` un número aleatorio de ese intervalo.

Nota: averigua el uso de la función `rand()` para generar números

b. Que la función tome una lista de n valores y devuelva el número menor y el número mayor de la lista

Nota: averigua el uso de las funciones: `array_push()`, `min()`, `max()`

c. Que la función tome una lista de n valores y devuelva un valor aleatorio de la lista

Nota: averigua el uso de las funciones y `array_rand()`

Actividades finales

1. Crea un formulario en HTML que recoja los siguientes datos del usuario:

- Nombre
- Apellidos
- Edad
- Dirección
- Teléfono
- Mail
- Ciclo
- Curso

Posteriormente crea un botón de envío para que al pulsar en él un programa PHP envíe los datos del formulario por el método GET.

El programa PHP debe almacenar los datos en una variable llamada `$datos_formulario` e imprimir todos los datos usando el bucle `foreach` ofreciendo la salida en forma de tabla HTML.

2. Crea un formulario en HTML con un input de cuadro de texto donde el usuario introduzca una contraseña.

Posteriormente crea un botón de envío para que al pulsar en él un programa PHP envíe la contraseña por el método POST.

El programa PHP debe comprobar que contraseña introducida es correcta:

- Si es correcta debe mostrar al usuario un mensaje de bienvenida
- Si es incorrecta vuelva a recargar la página para que el usuario vuelva a introducir la contraseña de nuevo